

FLOWER: Growing Agentic Computational Work into Reproducible Protocols

Zhanghao Zhouyin
McGill University

Abstract

Coding agents can now carry out substantial computational studies, but the record they leave is often harder to assess than the final result. Commands are tried in terminals, scripts are revised in place, long jobs outlive conversations, and the path from an initial question to a reported number is lost among abandoned attempts. Conventional workflow systems provide reliable execution once a pipeline has been specified, but exploratory work rarely begins with a finished pipeline. We present FLOWER, a lightweight, file-first workflow tool designed around this gap. A study begins immediately as an empty, inspectable run. The agent adds and revises steps while an append-only journal records attempts, results, decisions, and plan changes. Successful steps are stabilized as scripts with declared inputs, outputs, files, environments, and compute resources. At the end, FLOWER extracts the dependency closure behind a selected result as a re-runnable protocol with expected outputs. The same representation supports local processes, remote workstations, Slurm clusters, human gates, crash recovery, and a visual flow map. In this way, the exploratory history remains auditable while the final deliverable becomes smaller, deterministic, and reusable.

1 The missing object between a conversation and a pipeline

An agent asked to reproduce a scientific result does not normally know the final workflow in advance. It first finds data, tests an interpretation, writes a small program, discovers that a dependency is missing, changes the program, launches a larger calculation, and revises the analysis when the result exposes a mistake. This is not an exception to scientific work; it is how the method is found.

The difficulty is that the natural record of this process is a conversation plus a working directory. The conversation explains some decisions but is too long to audit as an execution trace. The directory contains the latest scripts but not necessarily the versions that produced the reported result. A scheduler knows that a job completed but not why the job was launched or which later conclusion depended on it. When an agent returns a number, a human reviewer must reconstruct these relationships by hand.

Mature workflow systems such as Snakemake [1] and Nextflow [2] make declared pipelines portable and repeatable. Provenance-oriented systems such as AiiDA [3] preserve rich graphs of calculations and data. These systems solve important parts of the reproducibility problem. The design point of FLOWER is earlier in the lifecycle: the period in which the workflow does not yet exist. Its central question is therefore not only “How should this pipeline run?” but also “How can the process of discovering the pipeline become the evidence for the final one?”

2 A study that grows

FLOWER treats a study as three related objects. A *plan* is the current human-readable description of the work. A *run* is the plan together with its immutable history. A *protocol* is the result-bearing part of a successful run, prepared for another person or machine to execute again.

The run is created before exploration begins. It may start with no nodes at all. Each computation that the agent relies on—including a download, a quick numerical check, or a failed pilot—is added as a step. A step states what it is meant to establish, what command it executes, which earlier results it uses, and which outputs or files it promises to produce. References to an earlier output also create dependency edges. A small plan can therefore remain close to the commands a researcher would have typed directly:

Listing 1: A minimal FLOWER plan.

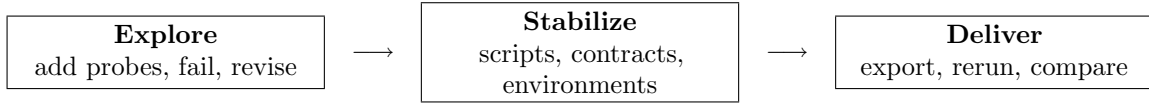


Figure 1: The run retains exploratory history, while protocol export keeps only the steps behind a selected result.

```

flower: 1
id: silicon-bands
title: Silicon band structure
nodes:
- id: relax
  kind: shell
  description: Relaxes the cell; energy is the convergence result.
  run: python3 relax.py > "$FLOWER_OUTPUTS"
  outputs: {energy: number, converged: boolean}

- id: bands
  kind: shell
  description: Computes and plots bands at the relaxed geometry.
  when: "${relax.outputs.converged}"
  run: python3 bands.py > "$FLOWER_OUTPUTS"
  files: {plot: bands.png}

```

This simplicity is deliberate. The agent remains outside the workflow engine and uses its existing coding environment. FLOWER does not provide an LLM loop, prompt language, or tool registry. The agent decides what to try; FLOWER executes the durable effect. Today the executable vocabulary is primarily a shell step and a human gate. Local programs, Python modules, simulation packages, and remote jobs all fit behind the same shell contract. This keeps the recorded workflow independent of a particular agent vendor or model.

Exploration does not require history to be rewritten. New steps and changes to unfinished steps become new plan generations. A change to completed work is explicit: the old attempt remains in the journal, the changed node is run again, and affected descendants become stale. Side studies and previews can remain in the run without contaminating the final protocol. They are evidence of how the method was found, not necessarily part of how the result must be reproduced.

3 Durability as a file format

The source of truth for a run is an append-only JSON Lines journal. Every event carries a contiguous sequence number, timestamp, actor label, and closed event type. Appends are serialized with a file lock and flushed to disk. Idempotency keys make retried writes harmless. A torn final line can be discarded after a crash, whereas corruption in the middle of the history is reported rather than guessed around. The visible run state is a pure fold over this journal; derived status files and views may be deleted and rebuilt.

Execution advances in short, re-entrant passes. A pass folds the journal, reconciles running work, records completed attempts, and starts newly ready nodes. Local commands run below detached supervisors. Remote work is either a detached process on a workstation or a batch job submitted to Slurm. Thus an agent session, UI, or foreground command does not need to remain alive while a calculation runs. A later status command can continue the same run.

Remote submission is treated as an irreversible boundary. Before submission, FLOWER records an intent with a deterministic key. Submission, remote markers, scheduler lookup, and an in-job ownership guard are arranged so that recovery favors reattachment over duplicate execution. Polling distinguishes scheduler delay, transport failure, preemption, node failure, memory exhaustion, timeout, cancellation, and a genuinely lost job. These distinctions matter because blindly repeating a scientific calculation is not always safe or useful.

Each successful attempt records two identities. The declaration hash describes what the node does, including the content of referenced local source files; the input hash describes resolved inputs and upstream output and file hashes. A result can be reused only when both identities match:

$$\text{reuse}(n) \iff h_{\text{decl}}^{\text{new}}(n) = h_{\text{decl}}^{\text{old}}(n) \wedge h_{\text{input}}^{\text{new}}(n) = h_{\text{input}}^{\text{old}}(n).$$

Scheduler resources and descriptive text are excluded from the declaration identity, so increasing memory or improving an explanation does not invalidate a scientific result. A deliberate rerun can still force the selected node while allowing unchanged descendants to be reused.

Software environments follow the same transition from exploration to stabilization. Commands used to explore a target machine can be collected into a small recipe consisting of setup, activation, and verification scripts. Freezing the recipe assigns it a content hash and snapshots its files. A fresh replay demonstrates that the recipe installs into an empty prefix. Changing the recipe produces a new prefix and changes the cache identity of every dependent step.

4 A flow map for human oversight

The journal is optimized for correctness, not for reading. FLOWER therefore projects it into a visual map of the current directed acyclic graph. Nodes show their purpose before their machinery: the description, result, declared files, attempts, logs, and failure details are available from the same view. A timeline presents the events in order, and a plan-history view shows each generation and amendment. Gates place a human decision in the graph itself rather than leaving approval in a separate conversation.

This human view is important because observability and reproducibility are different properties. A workflow may be technically replayable and still be difficult to judge. Requiring each step to say what it establishes gives the graph an explanatory layer. Conversely, the description alone is not trusted as execution history: the journal, output contracts, and file hashes record what actually happened. The model resembles the distinction in provenance standards between activities, entities, and responsible agents [4], but uses a compact, project-local representation during active work.

Human authority is represented through plan approvals, amendment decisions, and gate nodes. The current implementation records the declared actor associated with each action; it does not authenticate that identity. This is sufficient for collaboration on a trusted machine but not yet for institutional audit or regulatory use.

5 From a run to a protocol

When a result is ready, the user selects one or more successful nodes. FLOWER computes their ancestor closure and writes three files. `protocol.yaml` contains the selected nodes and everything they depend on; `expected.json` contains the structured results observed in the original run; and `PROTOCOL.md` explains the required inputs, machines, software recipes, steps, and reproduction command. Exploratory branches that do not contribute to the selected result disappear by construction.

The protocol is executed through the same engine as the exploratory run. Afterwards, a comparison command checks the new structured outputs against the recorded expectations, using numerical tolerances where appropriate. Protocol extraction is therefore not a separate packaging convention placed on top of the workflow. It is the final operation of the workflow model: historical evidence is reduced to a result-bearing recipe without erasing the original history.

6 Experience from computational science

The current implementation has been exercised through sixteen tracked reproductions spanning condensed matter physics, electronic structure, quantum chemistry, and molecular simulation. The workloads include exact diagonalization, quantum Monte Carlo, density-functional calculations, density-matrix renormalization, molecular dynamics, and coupled-cluster calculations. They run across the local machine, remote workstations, and Slurm-style execution, with fan-out campaigns ranging from small checks to tens of independent jobs.

These studies were used as design probes rather than as a single performance benchmark. They exposed failure modes that synthetic happy-path examples rarely reveal: a foreach campaign whose item list changes after an upstream rerun; a process that survives its driver; a scheduler submission whose reply is lost; a completed remote job whose result download temporarily fails; a checkpoint that should survive an automatic retry but not an intentional rerun; and a descriptive edit that should not repeat days of computation. The repository currently contains 359 automated test functions, including fake-scheduler fault injection and crash-window tests. More importantly, observed failures are retained as a bug diary and converted into regression tests.

The experiments also support the main usability claim. A study can begin before its final graph is known, grow while computations are running, and later yield a smaller protocol that has been rerun from a fresh checkout. The value of the tool is therefore not measured only by successful scheduling. It is measured by whether a person who was absent during the exploration can understand which steps support the final claim and can ask a new machine to perform them again.

7 Limitations and next steps

FLOWER is currently a project-local tool. Runs in separate clones are independent, actors are declared rather than authenticated, and there is no shared service for merging histories. The journal is crash-safe but not yet tamper-evident. Signed hash chains and authenticated approvals are natural extensions for results that must cross organizational boundaries.

Reproducibility also remains conditional on the quality of the recipe. FLOWER can guarantee that a frozen recipe is content-addressed and replayed consistently, but it cannot make an unpinned package installer deterministic. The present protocol comparison emphasizes structured JSON outputs; richer deliverables require per-output validation policies and explicit checks for large files, plots, and stochastic results. A future export manifest should bind the protocol to the exact source tree, environment recipes, declared artifacts, and git state that produced the selected results.

Finally, the current representation asks exploratory computations to become steps early. This is useful discipline, but lightweight probe events and an explicit “promote to stable step” operation could better represent the boundary between an observation used during exploration and a node intended for the final protocol.

8 Conclusion

Agentic computational work creates a new provenance problem: execution becomes faster while human reconstruction becomes harder. FLOWER addresses this problem by making the evolving workflow, rather than the conversation, the durable object. It allows a study to grow before its structure is known, preserves the failed and revised path by which that structure was found, and extracts the successful dependency closure as a re-runnable protocol. The result is a small bridge between open-ended agency and deterministic delivery: the agent is free to explore, the machine can recover, and the human receives both a readable flow map and a recipe that can be checked.

Availability. FLOWER is open-source software under the Apache License 2.0. The implementation and benchmark workflows are available at <https://github.com/floatingCatty/flower>.

References

- [1] J. Koster and S. Rahmann, “Snakemake—a scalable bioinformatics workflow engine,” *Bioinformatics*, vol. 28, no. 19, pp. 2520–2522, 2012. doi:10.1093/bioinformatics/bts480.
- [2] P. Di Tommaso, M. Chatzou, E. W. Floden, P. P. Barja, E. Palumbo, and C. Notredame, “Nextflow enables reproducible computational workflows,” *Nature Biotechnology*, vol. 35, pp. 316–319, 2017. doi:10.1038/nbt.3820.
- [3] S. P. Huber et al., “AiiDA 1.0, a scalable computational infrastructure for automated reproducible workflows and data provenance,” *Scientific Data*, vol. 7, article 300, 2020. doi:10.1038/s41597-020-00638-4.
- [4] L. Moreau and P. Missier, eds., “PROV-DM: The PROV Data Model,” W3C Recommendation, 30 April 2013. <https://www.w3.org/TR/prov-dm/>.